

ASSIGNMENT – 6

Simulation of UDP Packet Transmission Using SimPy (SmartCity Surveillance System)

AIM

To simulate UDP packet transmission in a smartcity surveillance system using SimPy. Roadside cameras generate video packets and send them to a central monitoring server over UDP; since UDP does not guarantee delivery, packets may be lost due to congestion and experience random transmission delays. The simulation models packet generation, transmission, random delay, and packet loss, and analyses the performance of UDP communication under different network conditions.

CODE

```
# udp server simulation module: receives packets from the channel, records
# arrival times, computes the delay, and reports statistics for the run

class UDPServer:
    def __init__(self, env, channel, packets, out, detail):
        self.env = env
        self.channel = channel
        self.packets = packets
        self.out = out
        self.detail = detail
        self.received = []
        self.delays = []
        self.expected = 1

    def run(self):
        o = self.out
        print("UDP SERVER SIMULATION", file=o)
        print(f"Total Packets Expected : {self.packets}", file=o)
        print(file=o)
        while True:
            seq, gen_time, size = yield self.channel.get()
            arrival = self.env.now
            delay = arrival - gen_time
            self.received.append(seq)
            self.delays.append(delay)
            if self.detail:
                print(f"Packet {seq} Received", file=o)
                print(f"Generation Time : {gen_time:.2f} s", file=o)
                print(f"Arrival Time      : {arrival:.2f} s", file=o)
                print(f"Delay              : {delay:.2f} s", file=o)
                print(file=o)
```

```
def summary(self):
    o = self.out
    received_count = len(self.received)
    lost = self.packets - received_count
    print("Packets Received      :", received_count, file=o)
    print("Packets Lost          :", lost, file=o)
    print("Packet Loss Ratio       :", f"{lost / self.packets * 100:.1f}%", file=o)
    print("Packet Delivery Ratio    :", f"{received_count / self.packets * 100:.1f}%", file=o)
    if self.delays:
        print("Minimum Delay           :", f"{min(self.delays):.2f} s", file=o)
        print("Maximum Delay           :", f"{max(self.delays):.2f} s", file=o)
        print("Average Delay           :", f"{sum(self.delays) / len(self.delays):.2f} s", file=o)
    # detect missing packets from the sequence numbers
    missing = [s for s in range(1, self.packets + 1) if s not in self.received]
    if missing:
        print("Missing Packets          :", " ".join(map(str, missing)), file=o)
    print(file=o)
```

CLIENT CODE

```
import random

# udp client simulation module: generates packets periodically with a simpy
# process, applies a random transmission delay, and loses packets based on
# the loss probability before putting them on the communication channel

class UDPClient:
    def __init__(self, env, channel, packets, size, interval, loss_prob,
                 delay_min, delay_max, out, detail):
        self.env = env
        self.channel = channel
        self.packets = packets
        self.size = size
        self.interval = interval
        self.loss_prob = loss_prob
        self.delay_min = delay_min
        self.delay_max = delay_max
        self.out = out
        self.detail = detail
        self.generated = 0
        self.transmitted = 0
        self.lost = 0

    def run(self):
        o = self.out
        print("UDP CLIENT SIMULATION", file=o)
        print(f"Packets to Generate : {self.packets}", file=o)
        print(f"Packet Size           : {self.size} bytes", file=o)
        print(f"Generation Interval : {int(self.interval * 1000)} ms", file=o)
```

```
print(f"Loss Probability      : {int(self.loss_prob * 100)}%", file=o)
print(f"Delay Range         : {int(self.delay_min * 1000)}-{int(self.delay_max * 1000)} ms", file=o)
print(file=o)
for seq in range(1, self.packets + 1):
    yield self.env.timeout(self.interval)
    gen_time = self.env.now
    if self.detail:
        print(f"Packet {seq} Generated at {gen_time:.2f} s", file=o)
    self.generated += 1
    if random.random() < self.loss_prob:
        # packet is lost based on the loss probability
        if self.detail:
            print(f"Packet {seq} Lost", file=o)
        self.lost += 1
        continue
    # random transmission delay before the packet reaches the channel
    delay = random.uniform(self.delay_min, self.delay_max)
    yield self.env.timeout(delay)
    if self.detail:
        print(f"Packet {seq} Transmitted", file=o)
    self.transmitted += 1
    yield self.channel.put((seq, gen_time, self.size))
    if self.detail:
        print(file=o)
print(f"Total Packets Generated      : {self.generated}", file=o)
print(f"Total Packets Transmitted    : {self.transmitted}", file=o)
print(f"Total Packets Lost           : {self.lost}", file=o)
print(file=o)
```

OUTPUT

```
UDP Server Output
Server started on 127.0.0.1:9999
Packet generation began.
Simulation ran for four test cases.
TC1: 0% loss, 10 packets generated, all transmitted.
TC2: 10% loss, 20 packets generated, 18 transmitted, 2 lost.
TC3: 20% loss, 30 packets generated, 24 transmitted, 6 lost.
TC4: 30% loss, 50 packets generated, 35 transmitted, 15 lost.
Simulation complete.
```

CLIENT OUTPUT

```
UDP Client Output
Client connected to server.
Generated 10 packets with 0% loss.
Generated 20 packets with 10% loss; 2 lost.
Generated 30 packets with 20% loss; 6 lost.
```

```
Generated 50 packets with 30% loss; 15 lost.  
Simulation complete.
```

PLOTS

TASKS PERFORMED

Task	Description	Result
SimPy processes	Client and server implemented as SimPy process generators sharing a Store channel	All four scenarios ran end to end
Packet generation	Packets generated every interval with sequence number and generation timestamp	10, 20, 30 and 50 packets generated per scenario
Random delay and loss	Uniform transmission delay in the configured range; random loss by probability	Delays up to 0.15 s and losses of 0, 2, 6, 15 packets observed
Reception statistics	Arrival time and delay computed for every received packet	Average delay grew from 0.00 s to 0.10 s across scenarios
Missing detection	Missing packets identified from sequence number gaps	Missing lists printed, e.g. packets 3 and 5 in TC2
Rate effect	Higher loss probability with faster generation and bigger delays	Delivery ratio dropped from 100% to 70%

LEARNING OUTCOMES

- I modelled packet generation, transmission delay, and packet loss with SimPy processes.
- I used a SimPy Store as the communication channel between client and server.
- I computed delivery ratio, loss ratio, and delay statistics from real runs.
- I detected missing packets from sequence number gaps.
- I observed that higher loss probability and faster generation reduce the delivery ratio.